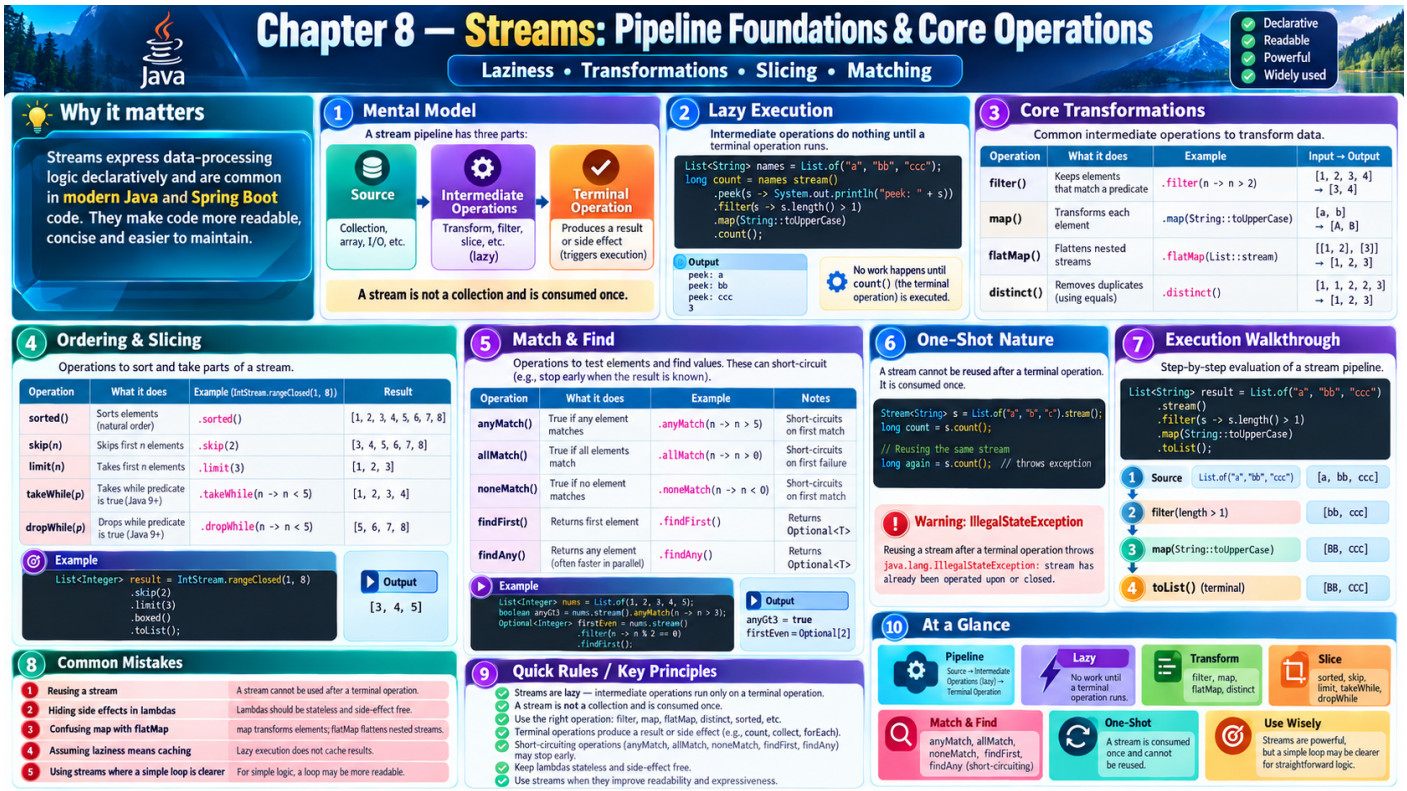
# CHAPTER 8 — STREAMS & OPTIONAL

Learn the mental model first. Then read complete programs, trace actual results, and recognize failure modes.

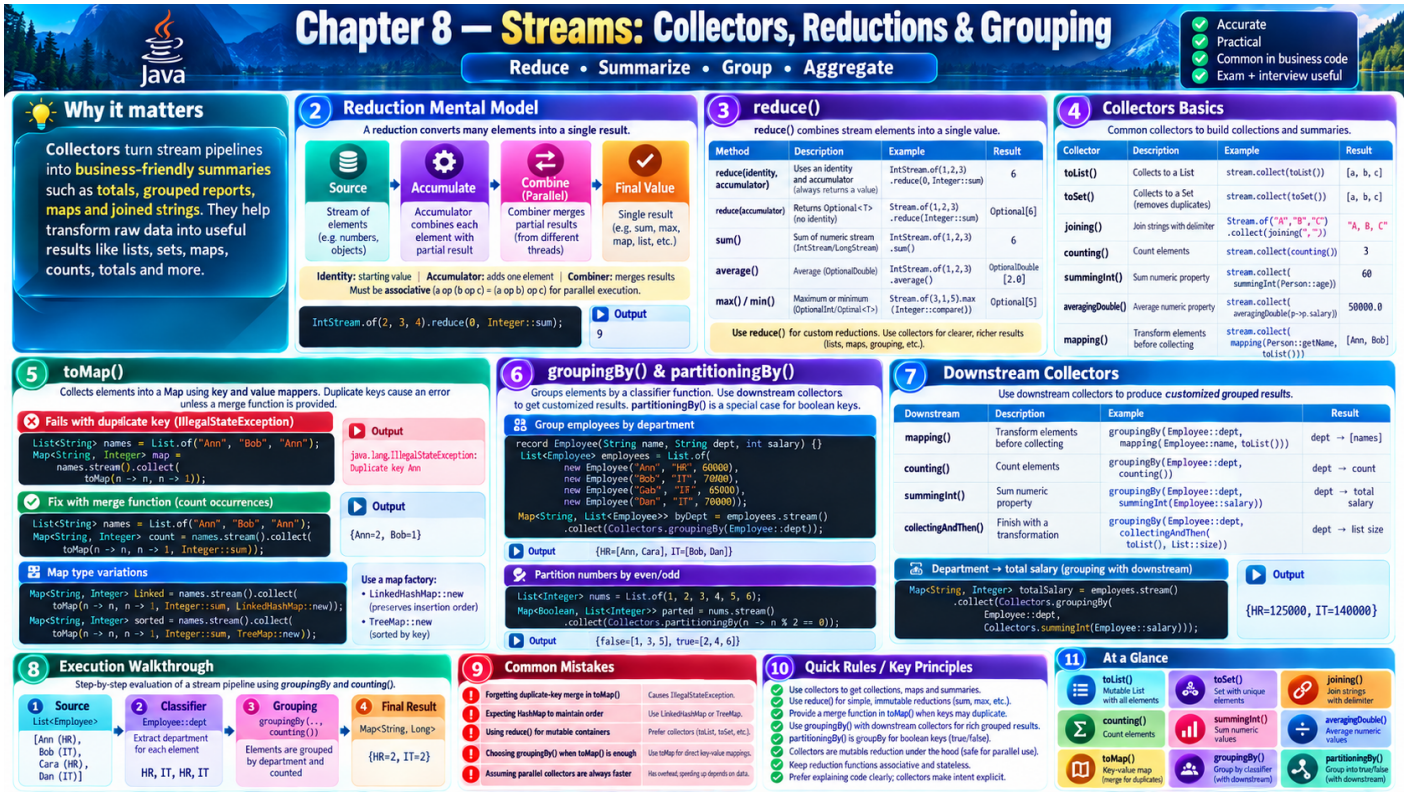
Java 17+ • concept-first • 10 complete programs • code and outputs • 3 glossy reference posters

## Infographic 1 — Overview



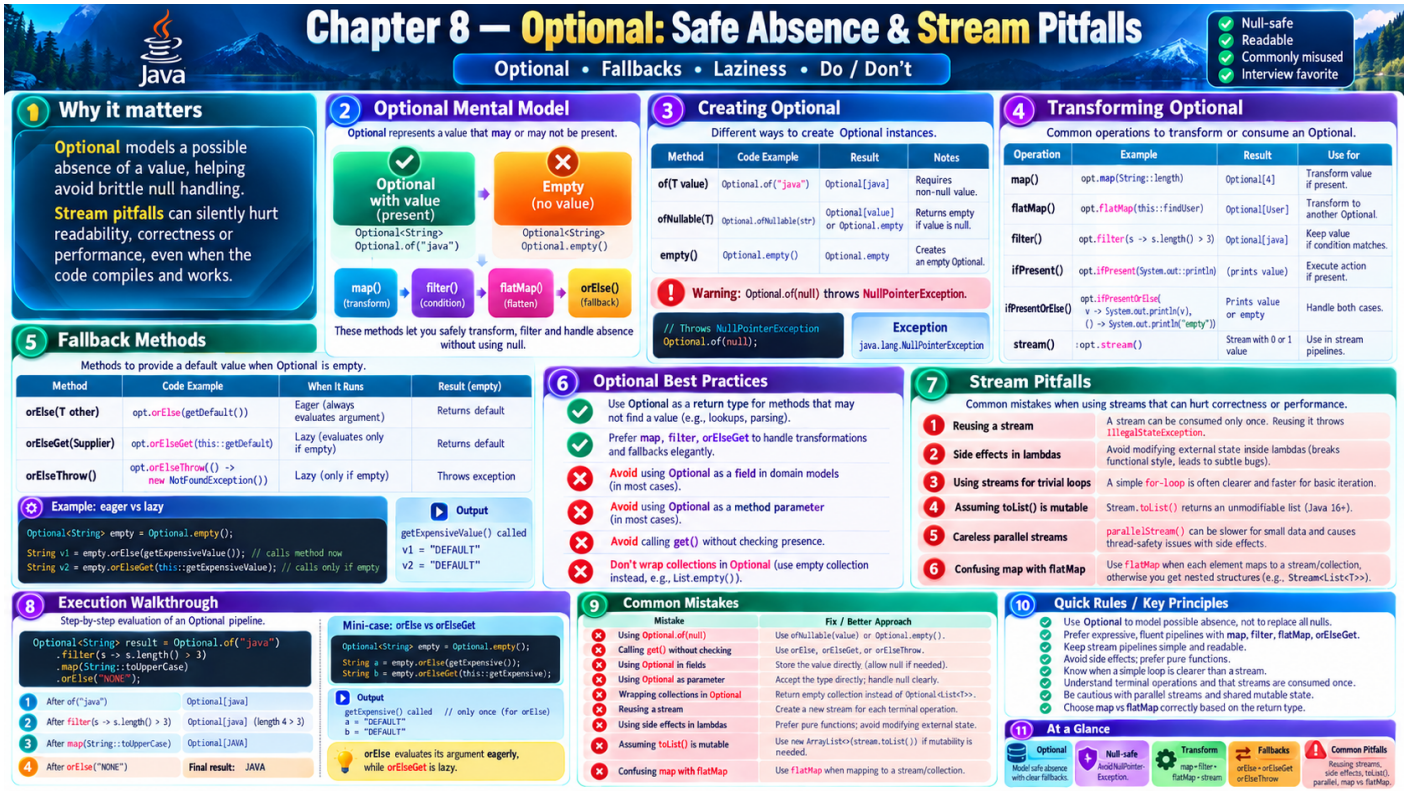

Use this poster as a quick review after studying the associated lessons. Full source files are authoritative for runnable code.

# Infographic 2 — Quick reference




Use this poster as a quick review after studying the associated lessons. Full source files are authoritative for runnable code.

# Infographic 3 — Quick reference




Use this poster as a quick review after studying the associated lessons. Full source files are authoritative for runnable code.

# How to read this chapter

Streams answer the question “How do I transform these values?” Optional answers “What if a value is missing?” The two ideas are often used together, but they solve different problems.

## Study sequence

- 1. Streams: what they are and how execution happens
- 2. Transformations: map, flatMap, distinct and mapMulti
- 3. Sorting, slicing and short-circuit tests
- 4. Reductions: identity, associativity and parallel results
- 5. Collectors: grouping, downstream aggregation and teeing
- 6. Maps from streams: duplicates, merges and null values
- 7. Optional: model absence and compose lookups
- 8. Optional.stream, empty-stream rules and failures
- 9. Parallel streams, state and performance
- 10. Complete business example: from orders to a report

Important distinction: a stream is not a database query engine. It generally processes values already accessible to your program. For a large dataset, ask whether filtering and aggregation should happen in SQL first.

# Lesson 1: Streams: what they are and how execution happens

A stream is a one-time description of work to perform on values. It is not a container that stores the resulting data. Build a pipeline from a source, add intermediate operations, then run it with a terminal operation.

### Build the mental model

• A source supplies elements, such as a List or an array.

• Intermediate operations such as filter and map define transformations, but normally do not run until needed.

• A terminal operation such as count, toList or findFirst starts traversal.

• A stream instance cannot be reused after a terminal operation. Create another stream from the source.

## Complete runnable program

```
package ch8;
import java.util.List;
public class PipelineAndLaziness
{
    public static void main(String[] args)
    {
        List<String> names = List.of("Amy", "Bob", "Brian", "Bea");
        var pipeline = names.stream()
            .filter(s -> { System.out.println("filter " + s); return s.startsWith("B"); })
            .map(s -> { System.out.println("map " + s); return s.toUpperCase(); });
        System.out.println("pipeline built");
        System.out.println("result=" + pipeline.findFirst().orElse("none"));
        // Streams are consumed after a terminal operation; never reuse pipeline.
    }
}
```

### Actual console output

```
pipeline built
filter Amy
filter Bob
map Bob
result=BOB
```

### Trace it: why this output occurred

• The program prints “pipeline built” before any filter output: creating intermediate steps is lazy.

• findFirst short-circuits: once it has a matching result, later elements do not need to be visited.

• The first element Amy fails the filter; Bob passes and becomes BOB; processing stops before Brian and Bea.

### Try it / check your understanding

• What lines print if findFirst() is replaced by toList()? Work through every element.

• Why is assigning stream.filter(...) not the same as assigning its output list?

**Practical rule:** Use streams for readable value transformations, not to hide a sequence of unrelated side effects.

# Lesson 1: deeper reasoning

Walk through the program without executing it. First identify the source and its element type. Next identify the functional interface each lambda implements. Then follow the values in encounter order. Finally locate the terminal operation and ask where an exception, short-circuit or side effect can change the path.

## What to examine in unfamiliar code

• Does the source have a defined encounter order? How does that affect output or correctness?

• What is the element type before and after each transformation? Write down the type after each stage.

• Which intermediate operations are stateless, which are stateful, and which terminal operations short-circuit?

• Are any values null or Optional.empty? Does the code intentionally handle absence?

• Are any captured variables or shared collections mutated? Would parallel execution still be correct?

• Could this work be delegated to the database or expressed more clearly with a loop?

Worked reasoning: The program prints “pipeline built” before any filter output: creating intermediate steps is lazy. findFirst short-circuits: once it has a matching result, later elements do not need to be visited. The first element Amy fails the filter; Bob passes and becomes BOB; processing stops before Brian and Bea.

Debugging tip: put a breakpoint inside a predicate or mapping function, inspect the current element, and compare the observed call sequence with the output shown on the previous page. Do not assume a pipeline executes one operation on every element before moving to the next.

## Review questions

• What lines print if findFirst() is replaced by toList()? Work through every element.

• Why is assigning stream.filter(...) not the same as assigning its output list?

Correction checkpoint: Use streams for readable value transformations, not to hide a sequence of unrelated side effects.

# Lesson 2: Transformations: map, flatMap, distinct and mapMulti

Mapping changes each element. Flattening turns a sequence of sequences into one sequence of values. This is common in business applications where an order has many line items or a department has many employees.

### Build the mental model

• map preserves one output element for each input element (unless the mapper throws).

• flatMap accepts a function returning a Stream and flattens all those inner streams.

• distinct removes duplicates according to equals and hashCode, preserving encounter order for ordered streams.

• mapMulti is a lower-allocation alternative when you emit zero or several values per input. Recognize it in modern Java.

## Complete runnable program

```
package ch8;
import java.util.List;
public class TransformAndFlatMap
{
    record Team(String name, List<String> members) { }
    public static void main(String[] args)
    {
        List<Team> teams = List.of(new Team("A", List.of("Mira", "Dee")),
                                   new Team("B", List.of("Sam", "Lee")));
        System.out.println("map=" + teams.stream().map(Team::name).toList());
        System.out.println("flatMap=" + teams.stream()
            .flatMap(team -> team.members().stream()).toList());
        System.out.println("distinct=" + List.of("a", "a", "b").stream().distinct().toList());
        System.out.println("mapMulti=" + teams.stream().<String>mapMulti(
            (team, sink) -> team.members().forEach(sink)).toList());
    }
}
```

### Actual console output

```
map=[A, B]
flatMap=[Mira, Dee, Sam, Lee]
distinct=[a, b]
mapMulti=[Mira, Dee, Sam, Lee]
```

### Trace it: why this output occurred

• map(Team::name) yields team names A and B, not team members.

• flatMap visits the member stream of team A then team B, yielding Mira, Dee, Sam, Lee.

• mapMulti sends the members to a downstream sink without requiring a new stream per team.

### Try it / check your understanding

• What is the difference in type between Stream> and Stream?

• When could flatMap yield fewer outputs than inputs? More outputs?

**Practical rule:** Use flatMap for nested lists and Optional chains; do not flatten away useful group boundaries accidentally.

# Lesson 2: deeper reasoning

Walk through the program without executing it. First identify the source and its element type. Next identify the functional interface each lambda implements. Then follow the values in encounter order. Finally locate the terminal operation and ask where an exception, short-circuit or side effect can change the path.

## What to examine in unfamiliar code

• Does the source have a defined encounter order? How does that affect output or correctness?

• What is the element type before and after each transformation? Write down the type after each stage.

• Which intermediate operations are stateless, which are stateful, and which terminal operations short-circuit?

• Are any values null or Optional.empty? Does the code intentionally handle absence?

• Are any captured variables or shared collections mutated? Would parallel execution still be correct?

• Could this work be delegated to the database or expressed more clearly with a loop?

Worked reasoning: map(Team::name) yields team names A and B, not team members. flatMap visits the member stream of team A then team B, yielding Mira, Dee, Sam, Lee. mapMulti sends the members to a downstream sink without requiring a new stream per team.

Debugging tip: put a breakpoint inside a predicate or mapping function, inspect the current element, and compare the observed call sequence with the output shown on the previous page. Do not assume a pipeline executes one operation on every element before moving to the next.

## Review questions

• What is the difference in type between Stream> and Stream?

• When could flatMap yield fewer outputs than inputs? More outputs?

Correction checkpoint: Use flatMap for nested lists and Optional chains; do not flatten away useful group boundaries accidentally.

# Lesson 3: Sorting, slicing and short-circuit tests

Sometimes a stream is a query over a collection: sort by business rules, skip and limit results, or test whether records meet a condition. The order of operations changes the result and performance.

### Build the mental model

• Comparator.comparingInt avoids unnecessary boxing compared with comparing for primitive fields.

• thenComparing gives a deterministic tie breaker.

• sorted is stateful and typically buffers elements; filter is usually cheaper before sorting.

• skip and limit depend on encounter order. takeWhile stops at the first failed predicate in an ordered stream.

• anyMatch, allMatch and noneMatch may stop early; allMatch and noneMatch return true for empty streams.

## Complete runnable program

```
package ch8;
import java.util.Comparator;
import java.util.List;
public class SortSliceAndMatch
{
    record Item(String name, int price) { }
    public static void main(String[] args)
    {
        List<Item> items = List.of(new Item("B", 50), new Item("A", 20),
                                   new Item("C", 50), new Item("D", 10));
        var sorted = items.stream()
            .sorted(Comparator.comparingInt(Item::price).reversed()
                .thenComparing(Item::name))
            .skip(1).limit(2).map(Item::name).toList();
        System.out.println("slice=" + sorted);
        System.out.println("any expensive=" + items.stream().anyMatch(i -> i.price() > 40));
        System.out.println("all positive=" + items.stream().allMatch(i -> i.price() > 0));
        System.out.println("none free=" + items.stream().noneMatch(i -> i.price() == 0));
        System.out.println("takeWhile=" + List.of(1, 2, 5, 3).stream()
            .takeWhile(n -> n < 4).toList());
    }
}
```

### Actual console output

```
slice=[C, A]
any expensive=true
all positive=true
none free=true
takeWhile=[1, 2]
```

### Trace it: why this output occurred

• Descending price sorts B and C first (both 50, tie broken alphabetically), then A and D.

• skip(1).limit(2) selects C and A.

• takeWhile on 1,2,5,3 gives 1,2 because the 5 stops traversal even though 3 would pass.

### Try it / check your understanding

- Predict what happens if limit(2) is moved before sorted().
- Explain why an empty stream satisfies allMatch() by definition.

**Practical rule:** For paginated database data, do not assume sorting a giant in-memory stream is a substitute for database ORDER BY and pagination.

# Lesson 3: deeper reasoning

Walk through the program without executing it. First identify the source and its element type. Next identify the functional interface each lambda implements. Then follow the values in encounter order. Finally locate the terminal operation and ask where an exception, short-circuit or side effect can change the path.

## What to examine in unfamiliar code

• Does the source have a defined encounter order? How does that affect output or correctness?

• What is the element type before and after each transformation? Write down the type after each stage.

• Which intermediate operations are stateless, which are stateful, and which terminal operations short-circuit?

• Are any values null or Optional.empty? Does the code intentionally handle absence?

• Are any captured variables or shared collections mutated? Would parallel execution still be correct?

• Could this work be delegated to the database or expressed more clearly with a loop?

Worked reasoning: Descending price sorts B and C first (both 50, tie broken alphabetically), then A and D. skip(1).limit(2) selects C and A. takeWhile on 1,2,5,3 gives 1,2 because the 5 stops traversal even though 3 would pass.

Debugging tip: put a breakpoint inside a predicate or mapping function, inspect the current element, and compare the observed call sequence with the output shown on the previous page. Do not assume a pipeline executes one operation on every element before moving to the next.

## Review questions

• Predict what happens if limit(2) is moved before sorted().

• Explain why an empty stream satisfies allMatch() by definition.

Correction checkpoint: For paginated database data, do not assume sorting a giant in-memory stream is a substitute for database ORDER BY and pagination.

# Lesson 4: Reductions: identity, associativity and parallel results

Reduction folds many values into one. For numbers this might be a sum or maximum; for objects it might be a combined business value. The operation must obey rules if parallel execution is possible.

### Build the mental model

• A valid identity e satisfies combine(e,x)=x and combine(x,e)=x.

• Associative operations may be regrouped without changing the result: (a op b) op c = a op (b op c).

• For three-argument reduce, accumulator and combiner must be compatible and associative.

• Do not mutate a shared collection inside reduce; use collect instead.

• Empty streams with no identity return Optional because there may be no answer.

## Complete runnable program

```
package ch8;
import java.util.List;
public class ReduceCorrectly
{
    public static void main(String[] args)
    {
        List<Integer> amounts = List.of(10, 20, 30);
        System.out.println("sum=" + amounts.stream().reduce(0, Integer::sum));
        System.out.println("max=" + amounts.stream().reduce(Integer::max).orElse(-1));
        System.out.println("parallel sum=" + amounts.parallelStream()
            .reduce(0, Integer::sum, Integer::sum));
        // Wrong: subtraction is not associative and cannot be combined safely.
        System.out.println("sequential subtraction=" + amounts.stream()
            .reduce(0, (a, b) -> a - b));
        System.out.println("parallel subtraction=" + amounts.parallelStream()
            .reduce(0, (a, b) -> a - b, (a, b) -> a - b));
    }
}
```

### Actual console output

```
sum=60
max=30
parallel sum=60
sequential subtraction=-60
parallel subtraction=-20
```

### Trace it: why this output occurred

• Sum uses identity 0 and addition, so 10+20+30=60.

• Maximum without an identity returns Optional(30).

• Subtraction is not associative: the parallel result can differ from sequential and is not a valid reduction.

### Try it / check your understanding

• Why is 1 an invalid identity for addition?

• For product multiplication, which identity is valid?

• Can you reduce a List by appending to the same ArrayList from the accumulator? Why not?

**Practical rule:** When debugging a mysterious parallel-stream total, check the identity and combiner before blaming the thread scheduler.

# Lesson 4: deeper reasoning

Walk through the program without executing it. First identify the source and its element type. Next identify the functional interface each lambda implements. Then follow the values in encounter order. Finally locate the terminal operation and ask where an exception, short-circuit or side effect can change the path.

## What to examine in unfamiliar code

• Does the source have a defined encounter order? How does that affect output or correctness?

• What is the element type before and after each transformation? Write down the type after each stage.

• Which intermediate operations are stateless, which are stateful, and which terminal operations short-circuit?

• Are any values null or Optional.empty? Does the code intentionally handle absence?

• Are any captured variables or shared collections mutated? Would parallel execution still be correct?

• Could this work be delegated to the database or expressed more clearly with a loop?

Worked reasoning: Sum uses identity 0 and addition, so 10+20+30=60. Maximum without an identity returns Optional(30). Subtraction is not associative: the parallel result can differ from sequential and is not a valid reduction.

Debugging tip: put a breakpoint inside a predicate or mapping function, inspect the current element, and compare the observed call sequence with the output shown on the previous page. Do not assume a pipeline executes one operation on every element before moving to the next.

## Review questions

• Why is 1 an invalid identity for addition?

• For product multiplication, which identity is valid?

• Can you reduce a List by appending to the same ArrayList from the accumulator? Why not?

Correction checkpoint: When debugging a mysterious parallel-stream total, check the identity and combiner before blaming the thread scheduler.

# Lesson 5: Collectors: grouping, downstream aggregation and teeing

Collectors build useful result structures: lists, sets, maps, group summaries and aggregates. The critical idea is a downstream collector: after choosing a group, you can apply another collector to values within that group.

### Build the mental model

• groupingBy(classifier) organizes elements using the classifier key.

• groupingBy(classifier, downstream) calculates a result separately for each group.

• Nested grouping produces a Map of Maps; use counting, mapping, filtering or reducing downstream.

• teeing runs two collectors on the same input and merges their results into one output.

• Collector characteristics, supplier, accumulator, combiner and finisher matter for performance and parallel correctness.

## Complete runnable program

```
package ch8;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;
public class CollectorsDeepDive
{
    record Sale(String region, String product, int amount) { }
    public static void main(String[] args)
    {
        var sales = List.of(new Sale("East", "A", 10), new Sale("West", "B", 20),
                            new Sale("East", "A", 30), new Sale("East", "B", 5));
        Map<String, Integer> sums = sales.stream().collect(Collectors.groupingBy(
            Sale::region, Collectors.summingInt(Sale::amount)));
        System.out.println("sums=" + new java.util.TreeMap<>(sums));
        var nested = sales.stream().collect(Collectors.groupingBy(Sale::region,
            Collectors.groupingBy(Sale::product, Collectors.counting())));
        System.out.println("east groups=" + new java.util.TreeMap<>(nested.get("East")));
        var stats = sales.stream().collect(Collectors.teeing(
            Collectors.counting(), Collectors.summingInt(Sale::amount),
            (count, total) -> "count=" + count + ", total=" + total));
        System.out.println(stats);
        System.out.println("names=" + sales.stream().map(Sale::product)
            .collect(Collectors.joining(",")));
    }
}
```

### Actual console output

```
sums={East=45, West=20}
east groups={A=2, B=1}
count=4, total=65
names=A,B,A,B
```

### Trace it: why this output occurred

• East sale amounts are 10,30,5, so East total is 45; West total is 20.

• East has two A records and one B record; nested grouping counts them.

• teeing counts four sales and sums 65; joining concatenates every product name.

## Try it / check your understanding

• Write the type of groupingBy(region, summingInt(amount)).

• Why should you not assume HashMap grouping order?

• When should you use collectingAndThen or mapping downstream?

**Practical rule:** For money, consider BigDecimal rather than floating-point primitive sums.

# Lesson 5: deeper reasoning

Walk through the program without executing it. First identify the source and its element type. Next identify the functional interface each lambda implements. Then follow the values in encounter order. Finally locate the terminal operation and ask where an exception, short-circuit or side effect can change the path.

## What to examine in unfamiliar code

• Does the source have a defined encounter order? How does that affect output or correctness?

• What is the element type before and after each transformation? Write down the type after each stage.

• Which intermediate operations are stateless, which are stateful, and which terminal operations short-circuit?

• Are any values null or Optional.empty? Does the code intentionally handle absence?

• Are any captured variables or shared collections mutated? Would parallel execution still be correct?

• Could this work be delegated to the database or expressed more clearly with a loop?

Worked reasoning: East sale amounts are 10,30,5, so East total is 45; West total is 20. East has two A records and one B record; nested grouping counts them. teeing counts four sales and sums 65; joining concatenates every product name.

Debugging tip: put a breakpoint inside a predicate or mapping function, inspect the current element, and compare the observed call sequence with the output shown on the previous page. Do not assume a pipeline executes one operation on every element before moving to the next.

## Review questions

• Write the type of groupingBy(region, summingInt(amount)).

• Why should you not assume HashMap grouping order?

• When should you use collectingAndThen or mapping downstream?

Correction checkpoint: For money, consider BigDecimal rather than floating-point primitive sums.

# Lesson 6: Maps from streams: duplicates, merges and null values

A stream is often converted into a Map keyed by an ID or code. But two records may share the same key. You must decide whether that represents an error, a legitimate aggregation or a business rule.

### Build the mental model

• Collectors.toMap(keyMapper, valueMapper) throws IllegalStateException when keys collide.

• Provide a merge function for duplicates: sum counts, pick the latest, or reject with a meaningful error.

• A four-argument toMap overload lets you choose a map implementation such as TreeMap.

• toUnmodifiableMap rejects null keys and values and duplicates without a merge rule.

• Stream.toList() produces an unmodifiable list; Collectors.toList() does not guarantee any particular mutability or implementation.

## Complete runnable program

```
package ch8;
import java.util.List;
import java.util.TreeMap;
import java.util.stream.Collectors;
public class ToMapAndDuplicates
{
    record Part(String sku, int qty) { }
    public static void main(String[] args)
    {
        var parts = List.of(new Part("X", 2), new Part("Y", 1), new Part("X", 5));
        try
        {
            parts.stream().collect(Collectors.toMap(Part::sku, Part::qty));
        }
        catch (IllegalStateException ex)
        {
            System.out.println("duplicate key detected");
        }
        var merged = parts.stream().collect(Collectors.toMap(Part::sku,
            Part::qty, Integer::sum, TreeMap::new));
        System.out.println("merged=" + merged);
        var immutable = parts.stream().map(Part::sku).distinct().toList();
        System.out.println("unmodifiable=" + immutable);
    }
}
```

### Actual console output

```
duplicate key detected
merged={X=7, Y=1}
unmodifiable=[X, Y]
```

### Trace it: why this output occurred

• Part X appears twice; the first toMap therefore fails and is caught.

• The second toMap merges X quantities 2 and 5 to obtain X=7; Y stays 1.

• TreeMap ensures sorted keys in the printed output.

## Try it / check your understanding

• What happens when a merge function returns null?

• Why might first-wins merging be dangerous for financial transactions?

**Practical rule:** Never silently drop duplicate business keys without a documented reason.

# Lesson 6: deeper reasoning

Walk through the program without executing it. First identify the source and its element type. Next identify the functional interface each lambda implements. Then follow the values in encounter order. Finally locate the terminal operation and ask where an exception, short-circuit or side effect can change the path.

## What to examine in unfamiliar code

• Does the source have a defined encounter order? How does that affect output or correctness?

• What is the element type before and after each transformation? Write down the type after each stage.

• Which intermediate operations are stateless, which are stateful, and which terminal operations short-circuit?

• Are any values null or Optional.empty? Does the code intentionally handle absence?

• Are any captured variables or shared collections mutated? Would parallel execution still be correct?

• Could this work be delegated to the database or expressed more clearly with a loop?

Worked reasoning: Part X appears twice; the first toMap therefore fails and is caught. The second toMap merges X quantities 2 and 5 to obtain X=7; Y stays 1. TreeMap ensures sorted keys in the printed output.

Debugging tip: put a breakpoint inside a predicate or mapping function, inspect the current element, and compare the observed call sequence with the output shown on the previous page. Do not assume a pipeline executes one operation on every element before moving to the next.

## Review questions

• What happens when a merge function returns null?

• Why might first-wins merging be dangerous for financial transactions?

Correction checkpoint: Never silently drop duplicate business keys without a documented reason.

# Lesson 7: Optional: model absence and compose lookups

Optional expresses that a return value may be absent. Instead of returning null and forcing every caller to guess, a method can return Optional and let the caller choose how to respond.

### Build the mental model

• Optional.of requires nonnull; ofNullable converts null into empty; empty creates absence.

• map transforms the present value; an empty Optional stays empty.

• flatMap is for a mapper that already returns Optional, avoiding Optional<Optional>.

• filter retains a present value only if it passes a predicate.

• orElse evaluates its fallback argument eagerly; orElseGet invokes its Supplier only if empty.

## Complete runnable program

```
package ch8;
import java.util.Map;
import java.util.Optional;
public class OptionalChains
{
    record User(String name, String email) { }
    static Optional<User> findUser(int id)
    {
        return Optional.ofNullable(Map.of(1, new User("Asha", "asha@example.org")).get(id));
    }
    static String expensiveDefault()
    {
        System.out.println("fallback evaluated");
        return "missing";
    }
    public static void main(String[] args)
    {
        System.out.println("email=" + findUser(1).map(User::email)
            .filter(email -> email.endsWith(".org")).orElse("none"));
        System.out.println("missing=" + findUser(9).map(User::name).orElse("unknown"));
        System.out.println("orElse=" + Optional.of("present").orElse(expensiveDefault()));
        System.out.println("orElseGet=" + Optional.of("present")
            .orElseGet(OptionalChains::expensiveDefault));
        System.out.println("flatMap=" + Optional.of(1).flatMap(OptionalChains::findUser)
            .map(User::name).orElse("none"));
    }
}
```

### Actual console output

```
email=asha@example.org
missing=unknown
fallback evaluated
orElse=present
orElseGet=present
flatMap=Asha
```

### Trace it: why this output occurred

• The successful lookup produces asha@example.org. The missing lookup uses unknown.

• Calling orElse(expensiveDefault()) prints fallback evaluated even though present was already available.

• The following orElseGet does not call the fallback. The nested flatMap lookup returns Asha.

## Try it / check your understanding

• Why should Optional usually not be a JPA entity field?

• What happens if you use Optional.of(null)?

• When is orElseThrow preferable to returning a default?

**Practical rule:** Prefer Optional as a return type for optional lookups, not as a blanket replacement for all nullable state.

# Lesson 7: deeper reasoning

Walk through the program without executing it. First identify the source and its element type. Next identify the functional interface each lambda implements. Then follow the values in encounter order. Finally locate the terminal operation and ask where an exception, short-circuit or side effect can change the path.

## What to examine in unfamiliar code

• Does the source have a defined encounter order? How does that affect output or correctness?

• What is the element type before and after each transformation? Write down the type after each stage.

• Which intermediate operations are stateless, which are stateful, and which terminal operations short-circuit?

• Are any values null or Optional.empty? Does the code intentionally handle absence?

• Are any captured variables or shared collections mutated? Would parallel execution still be correct?

• Could this work be delegated to the database or expressed more clearly with a loop?

Worked reasoning: The successful lookup produces asha@example.org. The missing lookup uses unknown. Calling orElse(expensiveDefault()) prints fallback evaluated even though present was already available. The following orElseGet does not call the fallback. The nested flatMap lookup returns Asha.

Debugging tip: put a breakpoint inside a predicate or mapping function, inspect the current element, and compare the observed call sequence with the output shown on the previous page. Do not assume a pipeline executes one operation on every element before moving to the next.

## Review questions

• Why should Optional usually not be a JPA entity field?

• What happens if you use Optional.of(null)?

• When is orElseThrow preferable to returning a default?

Correction checkpoint: Prefer Optional as a return type for optional lookups, not as a blanket replacement for all nullable state.

# Lesson 8: Optional.stream, empty-stream rules and failures

Optional and Stream connect cleanly. Optional.stream emits zero or one element. This is especially useful when mapping a collection of IDs to optional lookup results.

### Build the mental model

• map(this::lookup) creates Stream>; flatMap(Optional::stream) keeps only values that exist.

• Stream.empty is a valid stream. count is zero, anyMatch false, allMatch and noneMatch true.

• Optional.get() on empty throws NoSuchElementException; use orElseThrow with context if absence is an error.

• or, ifPresentOrElse and stream are useful Java 9+ operations.

• Never use Optional merely to hide the difference between required and optional data.

## Complete runnable program

```
package ch8;
import java.util.List;
import java.util.Optional;
public class OptionalStreamPitfalls
{
    static Optional<String> lookup(int id)
    {
        return id % 2 == 0 ? Optional.of("user" + id) : Optional.empty();
    }
    public static void main(String[] args)
    {
        System.out.println("present=" + List.of(1, 2, 3, 4).stream()
            .map(OptionalStreamPitfalls::lookup)
            .flatMap(Optional::stream).toList());
        System.out.println("empty count=" + List.<String>of().stream().count());
        System.out.println("empty allMatch=" + List.<String>of().stream()
            .allMatch(s -> s.length() > 10));
        System.out.println("nullable=" + Optional.ofNullable(null).isEmpty());
        try
        {
            Optional.empty().orElseThrow(() -> new IllegalArgumentException("not found"));
        }
        catch (IllegalArgumentException ex)
        {
            System.out.println("caught=" + ex.getMessage());
        }
    }
}
```

### Actual console output

```
present=[user2, user4]
empty count=0
empty allMatch=true
nullable=true
caught=not found
```

### Trace it: why this output occurred

• IDs 2 and 4 produce values, while 1 and 3 disappear from the flattened stream.

• The empty stream has allMatch=true because there is no counterexample.

• An explicitly required missing value triggers IllegalArgumentException and is handled.

## Try it / check your understanding

• Rewrite nested Optional> using flatMap.

• Explain why empty allMatch=true is mathematically consistent.

**Practical rule:** For cross-layer application code, carry a meaningful exception when absence violates a business rule.

# Lesson 8: deeper reasoning

Walk through the program without executing it. First identify the source and its element type. Next identify the functional interface each lambda implements. Then follow the values in encounter order. Finally locate the terminal operation and ask where an exception, short-circuit or side effect can change the path.

## What to examine in unfamiliar code

• Does the source have a defined encounter order? How does that affect output or correctness?

• What is the element type before and after each transformation? Write down the type after each stage.

• Which intermediate operations are stateless, which are stateful, and which terminal operations short-circuit?

• Are any values null or Optional.empty? Does the code intentionally handle absence?

• Are any captured variables or shared collections mutated? Would parallel execution still be correct?

• Could this work be delegated to the database or expressed more clearly with a loop?

Worked reasoning: IDs 2 and 4 produce values, while 1 and 3 disappear from the flattened stream. The empty stream has allMatch=true because there is no counterexample. An explicitly required missing value triggers IllegalArgumentException and is handled.

Debugging tip: put a breakpoint inside a predicate or mapping function, inspect the current element, and compare the observed call sequence with the output shown on the previous page. Do not assume a pipeline executes one operation on every element before moving to the next.

## Review questions

• Rewrite nested Optional> using flatMap.

• Explain why empty allMatch=true is mathematically consistent.

Correction checkpoint: For cross-layer application code, carry a meaningful exception when absence violates a business rule.

# Lesson 9: Parallel streams, state and performance

Parallel streams can use multiple threads to process independent parts of a source. They are not automatically faster, and side effects can be unsafe if multiple threads mutate shared state.

### Build the mental model

• Use stateless, non-interfering operations for stream pipelines.

• Never write to an ordinary ArrayList from parallel forEach without synchronization.

• An AtomicInteger avoids lost updates, but a built-in reduction is usually simpler and faster.

• Encounter order differs from execution order; forEach may print in arbitrary order, while forEachOrdered preserves encounter order at a cost.

• Benchmark realistic workloads; blocking I/O and small collections are often poor fits for parallel streams.

## Complete runnable program

```
package ch8;
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;
public class ParallelAndSideEffects
{
    public static void main(String[] args)
    {
        var values = java.util.stream.IntStream.rangeClosed(1, 1000).boxed().toList();
        int sum = values.parallelStream().mapToInt(Integer::intValue).sum();
        System.out.println("safe sum=" + sum);
        AtomicInteger atomic = new AtomicInteger();
        values.parallelStream().forEach(atomic::addAndGet);
        System.out.println("atomic sum=" + atomic.get());
        System.out.println("ordered=" + List.of(3, 1, 2).parallelStream()
            .sorted().toList());
        // Unsafe: writing to an ordinary ArrayList from parallel forEach.
        // Prefer collect/toList or built-in reductions; atomic only for illustration.
    }
}
```

### Actual console output

```
safe sum=500500
atomic sum=500500
ordered=[1, 2, 3]
```

### Trace it: why this output occurred

• The built-in primitive sum computes 500500 reliably.

• AtomicInteger computes the same number correctly although the order of additions can differ.

• sorted().toList() returns [1,2,3] in defined encounter order.

### Try it / check your understanding

• Why can synchronizedList still have iteration hazards?

• What makes a source spliterator easier to parallelize?

• Why is a parallel HTTP-call stream often a poor design?

**Practical rule:** In server apps, parallel streams use the common ForkJoinPool by default; recognize their interaction with shared workloads.

# Lesson 9: deeper reasoning

Walk through the program without executing it. First identify the source and its element type. Next identify the functional interface each lambda implements. Then follow the values in encounter order. Finally locate the terminal operation and ask where an exception, short-circuit or side effect can change the path.

## What to examine in unfamiliar code

• Does the source have a defined encounter order? How does that affect output or correctness?

• What is the element type before and after each transformation? Write down the type after each stage.

• Which intermediate operations are stateless, which are stateful, and which terminal operations short-circuit?

• Are any values null or Optional.empty? Does the code intentionally handle absence?

• Are any captured variables or shared collections mutated? Would parallel execution still be correct?

• Could this work be delegated to the database or expressed more clearly with a loop?

Worked reasoning: The built-in primitive sum computes 500500 reliably. AtomicInteger computes the same number correctly although the order of additions can differ. sorted().toList() returns [1,2,3] in defined encounter order.

Debugging tip: put a breakpoint inside a predicate or mapping function, inspect the current element, and compare the observed call sequence with the output shown on the previous page. Do not assume a pipeline executes one operation on every element before moving to the next.

## Review questions

• Why can synchronizedList still have iteration hazards?

• What makes a source spliterator easier to parallelize?

• Why is a parallel HTTP-call stream often a poor design?

Correction checkpoint: In server apps, parallel streams use the common ForkJoinPool by default; recognize their interaction with shared workloads.

# Lesson 10: Complete business example: from orders to a report

In a Spring-style business feature, you may receive orders, select eligible ones, expand their line items, calculate money and aggregate quantities. This complete program integrates the techniques in one place.

### Build the mental model

• First, filter to PAID orders.

• Then flatMap each selected order into its line items.

• Use BigDecimal and reduce for money amounts.

• Use toMap with a merge function for product totals.

• Finish by sorting entries to choose the top quantity SKU with deterministic tie breaking.

## Complete runnable program

```
package ch8;
import java.math.BigDecimal;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;
import java.util.stream.Collectors;
public class EndToEndOrders
{
    record Line(String sku, int quantity, BigDecimal unitPrice) { }
    record Order(String region, String status, List<Line> lines) { }
    public static void main(String[] args)
    {
        var orders = List.of(
            new Order("East", "PAID", List.of(new Line("A", 2, new BigDecimal("10.00")),
                new Line("B", 1, new BigDecimal("5.00")))),
            new Order("West", "OPEN", List.of(new Line("A", 1, new BigDecimal("10.00")))),
            new Order("East", "PAID", List.of(new Line("C", 3, new BigDecimal("7.00")))));
        var paidLines = orders.stream().filter(o -> o.status().equals("PAID"))
            .flatMap(o -> o.lines().stream()).toList();
        var gross = paidLines.stream().map(line -> line.unitPrice()
            .multiply(BigDecimal.valueOf(line.quantity())))
            .reduce(BigDecimal.ZERO, BigDecimal::add);
        var quantities = paidLines.stream().collect(Collectors.toMap(
            Line::sku, Line::quantity, Integer::sum, TreeMap::new));
        System.out.println("gross=" + gross);
        System.out.println("quantities=" + quantities);
        System.out.println("top SKU=" + quantities.entrySet().stream()
            .sorted(Map.Entry.<String, Integer>comparingByValue(Comparator.reverseOrder())
                .thenComparing(Map.Entry.comparingByKey()))
            .map(Map.Entry::getKey).findFirst().orElse("none"));
    }
}
```

### Actual console output

```
gross=46.00
quantities={A=2, B=1, C=3}
top SKU=C
```

## Trace it: why this output occurred

• The West OPEN order is excluded.

• Paid order 1 contributes A=2 and B=1; paid order 3 contributes C=3.

• Gross = 2*10.00+1*5.00+3*7.00 = 46.00.

• Top SKU is C with quantity 3.

## Try it / check your understanding

• Where would transaction handling belong in a real service?

• How would you avoid loading millions of orders into memory?

• How would you refactor calculation into testable named functions?

**Practical rule:** Use this example as the bridge to reading business service code. Streams transform data; they do not replace validation, transactions, tests or architecture.

# Lesson 10: deeper reasoning

Walk through the program without executing it. First identify the source and its element type. Next identify the functional interface each lambda implements. Then follow the values in encounter order. Finally locate the terminal operation and ask where an exception, short-circuit or side effect can change the path.

## What to examine in unfamiliar code

• Does the source have a defined encounter order? How does that affect output or correctness?

• What is the element type before and after each transformation? Write down the type after each stage.

• Which intermediate operations are stateless, which are stateful, and which terminal operations short-circuit?

• Are any values null or Optional.empty? Does the code intentionally handle absence?

• Are any captured variables or shared collections mutated? Would parallel execution still be correct?

• Could this work be delegated to the database or expressed more clearly with a loop?

Worked reasoning: The West OPEN order is excluded. Paid order 1 contributes A=2 and B=1; paid order 3 contributes C=3. Gross = 2*10.00+1*5.00+3*7.00 = 46.00. Top SKU is C with quantity 3.

Debugging tip: put a breakpoint inside a predicate or mapping function, inspect the current element, and compare the observed call sequence with the output shown on the previous page. Do not assume a pipeline executes one operation on every element before moving to the next.

## Review questions

• Where would transaction handling belong in a real service?

• How would you avoid loading millions of orders into memory?

• How would you refactor calculation into testable named functions?

Correction checkpoint: Use this example as the bridge to reading business service code. Streams transform data; they do not replace validation, transactions, tests or architecture.

# Audit-driven worked cases: solve before revealing answers

These ten cases repair the biggest weakness in the initial draft: they are specific to the Java behavior rather than generic instructions repeated after every lesson.

## Execution order: filter + map + findFirst

Given List.of("A", "BB", "CCC").stream().filter(s -> s.length() > 1).map(String::length).findFirst(), the result is Optional[2]. The source A is rejected, BB is retained and mapped to 2, and CCC is never evaluated. Do not think of the pipeline as a full list filtered first, then transformed.

## Encounter-order warning

For an ordered stream, findFirst returns the earliest matching value. findAny can return any matching value, particularly when parallel. sorted restores an ordering but buffers values. limit before sorted means sort a subset; sorted before limit means choose the best subset.

## Three-stage type trace

If Order has List lines, orders.stream() has type Stream; filter retains Stream; flatMap(o -> o.lines().stream()) changes this to Stream; map(Line::sku) yields Stream; distinct().toList() returns List.

## Reduction proof: addition vs subtraction

Addition with identity 0 is valid: (10+20)+30 equals 10+(20+30). Subtraction fails: (10-20)-30 equals -40, while 10-(20-30) equals 20. That difference means parallel repartitioning can change a result. A parallel reduction with subtraction is not reliable.

## Collector life cycle

For groupingBy(region, summingInt(amount)), the classifier chooses a group, the downstream summing collector accumulates amounts for that group, and map merging combines partial groups in parallel. Think in terms of supplier, accumulator, combiner and optional finisher; not just a magic mapping function.

## toMap duplicate decision

Given Part(X,2), Part(Y,1), Part(X,5), toMap without a merge rule fails on X. toMap(key, qty, Integer::sum) produces X=7, Y=1. Choosing (a,b)->a silently discards the second X and may conceal a business-data defect.

## Optional fallback evaluation

Optional.of("ok").orElse(expensive()) evaluates expensive() before calling orElse, even though it ignores the result. Optional.of("ok").orElseGet(() -> expensive()) does not call expensive at all. Use orElse for cheap already-computed values and orElseGet for deferred work.

## Nested optional types

Optional.map(u -> findManager(u)) produces Optional> if findManager returns Optional. Use flatMap(u -> findManager(u)) to produce Optional. In streams, a Stream> can be flattened with flatMap(Optional::stream).

## Parallel race diagnosis

List result = new ArrayList<>(); stream.parallel().forEach(result::add) is unsafe: ArrayList mutation is not synchronized. The result may be incomplete or corrupted. Prefer stream.parallel().map(...).toList(), a collector, or an explicit concurrent structure only when necessary.

## Order-report sanity check

PAID orders contribute A: 2x10.00 = 20.00, B: 1x5.00 = 5.00 and C: 3x7.00 = 21.00. The gross is 46.00, quantities are A=2, B=1, C=3 and top SKU is C. Writing these intermediate values down catches many bugs before coding.

# Advanced recognition: operations you will meet in repositories

### Primitive streams

IntStream, LongStream and DoubleStream avoid boxing for numeric work. Use mapToInt then sum, average or summaryStatistics. Beware overflow with int sums.

### Collectors.toList vs Stream.toList

Stream.toList returns an unmodifiable list and may be used on Java 16+. Collectors.toList makes no guarantee about mutability or implementation. List.copyOf creates an unmodifiable snapshot of its elements, not a deep copy.

### Collector internals

The supplier creates a result container, accumulator adds values, combiner merges containers, and finisher produces the result. The identity/associativity constraints are crucial for parallel correctness.

### Noninterference

Do not structurally modify the stream source while traversing it (unless documented for the source). Keep mapping/filtering functions stateless for predictability.

### peek and debugging

peek is mainly for diagnosis. Optimization and short circuiting may mean not every element is observed. Do not put required business side effects inside peek.

### Exception behavior

Checked exceptions do not fit directly into many standard functional interfaces. Prefer a clear helper that handles or translates exceptions, rather than swallowing them in a lambda.

### Null and Optional

Do not assume map() turns null into Optional: Optional.map can return empty if the mapper returns null, but Stream.map permits null results. Optional is not a replacement for validation.

### Parallel and ordering

parallelStream is a trade-off, not a free improvement. Stateful stages, ordered operations, common-pool contention, and workload size determine whether it helps.

### flatMap and resource ownership

Stream.flatMap closes mapped streams as part of flattening, but streams opened from files need proper try-with-resources ownership at the outer boundary.

### Optional API discipline

Use Optional for return-value absence and avoid using isPresent/get in place of map, flatMap, orElseThrow. Keep meaningful domain failures explicit.

# 9/10 depth checklist — self-test

• Explain a stream pipeline’s lazy execution and the difference between intermediate and terminal stages.

• Predict filter-map-findFirst execution order and why some lambdas are never called.

• Show the difference between map, flatMap and mapMulti on nested business objects.

• Explain stateful sorted/distinct operations, slicing and encounter-order constraints.

• Predict anyMatch, allMatch and noneMatch on empty sources.

• Choose a mathematically valid reduce identity and associative combiner.

• Build nested groupingBy with downstream counting/summing and a teeing collector.

• Diagnose duplicate keys in toMap and use explicit merge semantics.

• Explain Optional.map, flatMap, filter, or, stream, orElse vs orElseGet.

• Spot parallel-stream races, unsafe side effects, and common pool performance risks.

• Read an end-to-end business pipeline and translate it to a readable imperative loop.

• Decide when a loop, SQL query, or a stream is clearest for the task.

If you can explain all twelve without guessing—and predict the ten outputs—you have the practical understanding expected from this chapter.

# Chapter 9

# Annotations, Reflection & Generated Code

**A concept-first, runnable-code refresher • Java 17+**

You already know ordinary Java syntax. This chapter focuses on a different problem: understanding the behavior that is **not obvious from normal handwritten method calls**. Annotations label code; reflection examines code structure while an application runs; code generators create source or bytecode you may never have written yourself. These mechanisms are common in Spring Boot, serializers, ORMs, dependency injection, and test frameworks.

**What you will learn**

- Read annotations and predict which tools can see them.
- Use reflection to inspect classes and invoke a method safely.
- Understand the difference between generated source, compiler-supplied members, and runtime proxies.
- Debug hidden behavior without mistaking a marker annotation for executable logic.
- Trace a complete example, then run each included Java file.

# Chapter map and learning approach

• Part A — Annotations: meaning, syntax, targets, retention, inheritance, repeatable annotations, runtime scanning, processors.

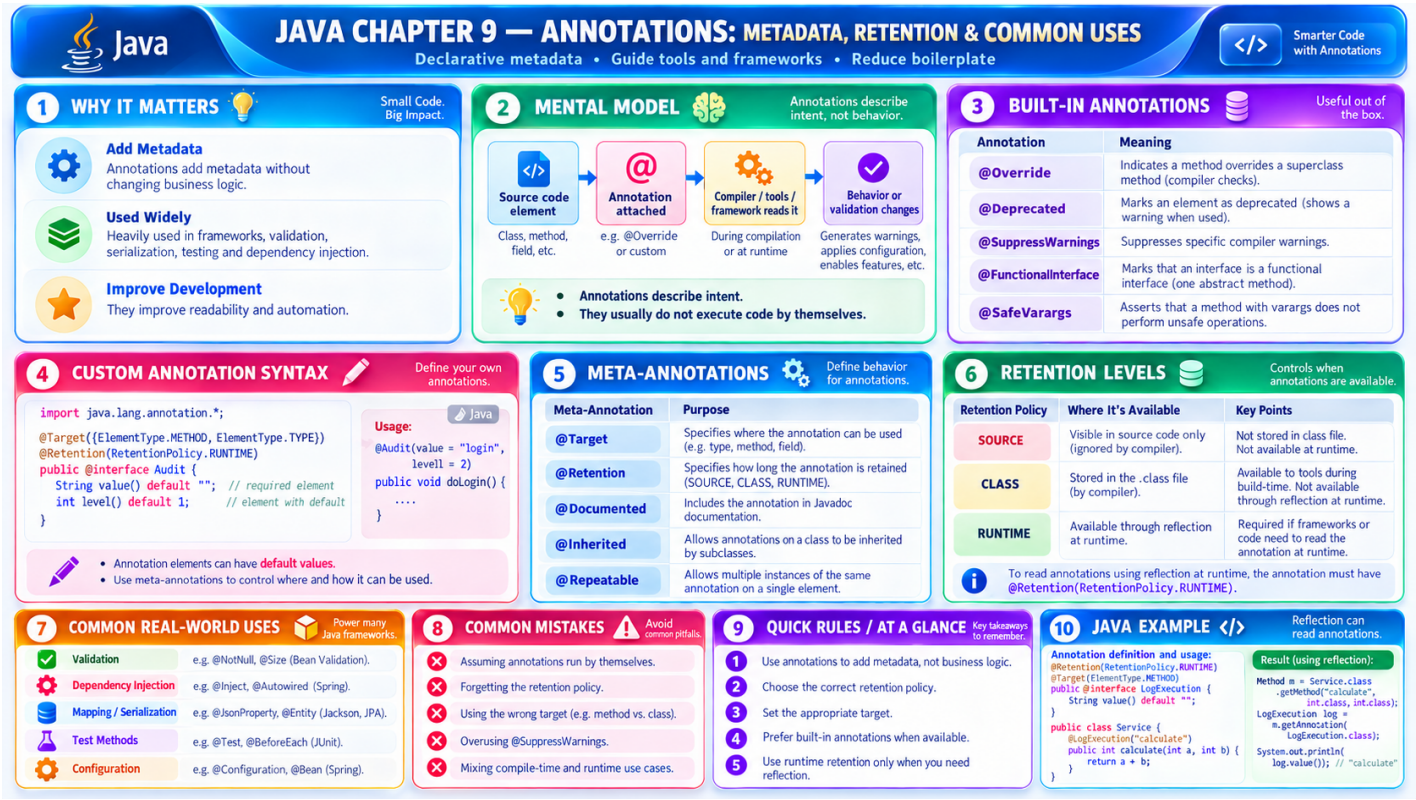
• Part B — Reflection: Class objects, fields and methods, access checks, generic metadata, invocation failures, and module boundaries.

• Part C — Generated Code: records, Lombok, annotation processing, proxies, generated sources, and framework connections.

• Final audit: application checklist, case study, source files, traps, and worked exercises.

Every lesson begins with a simple purpose, then shows the behavior, a verified complete program where useful, a failure mode, and a worked question. The code is organized into one small package so you can run the examples in IntelliJ or from the command line.

# PART A — ANNOTATIONS



## A1. What an annotation actually is

### The idea in ordinary English

- An annotation is structured metadata attached to a class, method, field, parameter, or another allowed element.
- An annotation does not execute a method merely because it appears above that method. A compiler, processor, framework, or reflection-based scanner must choose to interpret it.

For example, `@Override` asks the Java compiler to check an override; `@Action` only does something if your application has code that reads it.

### Follow the behavior step by step

- Declare an annotation type with `@interface`.
- Choose allowed locations with `@Target`.
- Choose visibility lifetime with `@Retention`.
- Build a scanner or processor that reads the annotation and takes action.

### What can go wrong

- Expecting any custom annotation to perform work automatically.
- Assuming all annotations are visible through reflection.

### Challenge and worked answer

- **Question:** If you add @Action to a method but never scan for @Action, does the method run?
- **Answer:** No. A custom annotation describes metadata; a reader must decide what to do with it.

## A2. Creating and reading a runtime annotation

### The idea in ordinary English

- You can define annotation elements such as value(), priority(), and defaults.
- When retention is RUNTIME, reflection can discover these values from the compiled class.

This complete example labels one method as an action and leaves another unannotated. The scanner inspects declared methods, finds the marker, prints its values, and invokes only the marked method.

### Follow the behavior step by step

- `getDeclaredMethods()` enumerates declared methods; the order is unspecified.
- `getAnnotation(Action.class)` returns metadata or null.
- The scanner prints its elements before invoking the selected method.
- The result is two lines: action metadata and method output.

### Complete source: AnnotationDemo.java

```
package demo.ch9;
import java.lang.annotation.*;
import java.lang.reflect.*;
@Retention(RetentionPolicy.RUNTIME)
```

```
@Target(ElementType.METHOD)
@interface Action { String value(); int priority() default 1; }
class Jobs {
    @Action(value="IMPORT", priority=5)
    public void importData() { System.out.println("Import executed"); }
    public void skip() { System.out.println("Unannotated"); }
}
public class AnnotationDemo {
    public static void main(String[] args) throws Exception {
        Jobs jobs = new Jobs();
        for (Method m : Jobs.class.getDeclaredMethods()) {
            Action a = m.getAnnotation(Action.class);
            if (a != null) {
                System.out.println(a.value()+" / priority="+a.priority());
                m.invoke(jobs);
            }
        }
    }
}
```

## Verified execution / build result

```
IMPORT / priority=5
Import executed
```

A design note: a production scanner should validate that the method accepts the required parameters and has the expected return type; it should report configuration errors clearly.

### What can go wrong

- Never rely on reflection method iteration order.
- Do not invoke annotated methods without checking signatures and permissions.

### Challenge and worked answer

- **Question:** What would happen if you changed Action retention to CLASS?
- **Answer:** The annotation remains in the .class file but normal runtime reflection no longer returns it.

# A3. Retention, target, inheritance and repeatable metadata

**The idea in ordinary English**

- SOURCE metadata is discarded by the compiler; CLASS metadata goes into bytecode; RUNTIME can be read via reflection.
- @Inherited is limited: it affects class-level annotations obtained through class inheritance, not every kind of annotated element.
- @Repeatable allows multiple instances of the same annotation on one target.

Think of these controls as metadata reach and shape. Choosing them correctly affects whether a processor, runtime scanner, or subclass sees the labels.

**Follow the behavior step by step**

- Look at ChildJob.class.getAnnotation(Role.class): the superclass role is inherited because Role uses @Inherited.
- Look at Batch.class.getAnnotationsByType(Tag.class): it resolves repeated @Tag instances.
- Note that repeating annotations requires the declared container @Tags with the proper value() method.

## Complete source: AnnotationRulesDemo.java

```
package demo.ch9;
import java.lang.annotation.*;
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
@Inherited
@interface Role { String value(); }
@Role("Base") class ParentJob { }
class ChildJob extends ParentJob { }
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
@Repeatable(Tags.class)
@interface Tag { String value(); }
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
@interface Tags { Tag[] value(); }
@Tag("ETL") @Tag("NIGHTLY") class Batch { }
public class AnnotationRulesDemo {
    public static void main(String[] args) {
        Role role = ChildJob.class.getAnnotation(Role.class);
        System.out.println("Inherited role=" + role.value());
        for (Tag tag: Batch.class.getAnnotationsByType(Tag.class)) {
            System.out.println("Tag=" + tag.value());
        }
    }
}
```

## Verified execution / build result

```
Inherited role=Base
Tag=ETL
Tag=NIGHTLY
```

**What can go wrong**

- Assuming @Inherited applies to interfaces or to method annotations.
- Using @Retention(SOURCE) for runtime scanning.

**Challenge and worked answer**

- **Question:** Does @Inherited mean a class inherits annotations declared on its implemented interface?
- **Answer:** No. The special inheritance lookup applies to superclass declarations for class-level annotations.

## A4. Compile-time processing versus runtime scanning

### The idea in ordinary English

- Annotation processors run during compilation and can generate source files or report problems.
- Runtime scanners inspect metadata when an application is executing.
- They are different phases, with different tools and limitations.

Common compile-time examples include MapStruct and Lombok-related tooling; runtime use is common for reflective discovery. Not every framework annotation is handled at runtime by reflection alone.

### Follow the behavior step by step

- Compiler sees source annotations in processing rounds.
- Processor can inspect annotated elements and generate new sources using Filer.
- Java compiler processes newly generated sources in later rounds.
- The generated class is compiled along with handwritten classes.

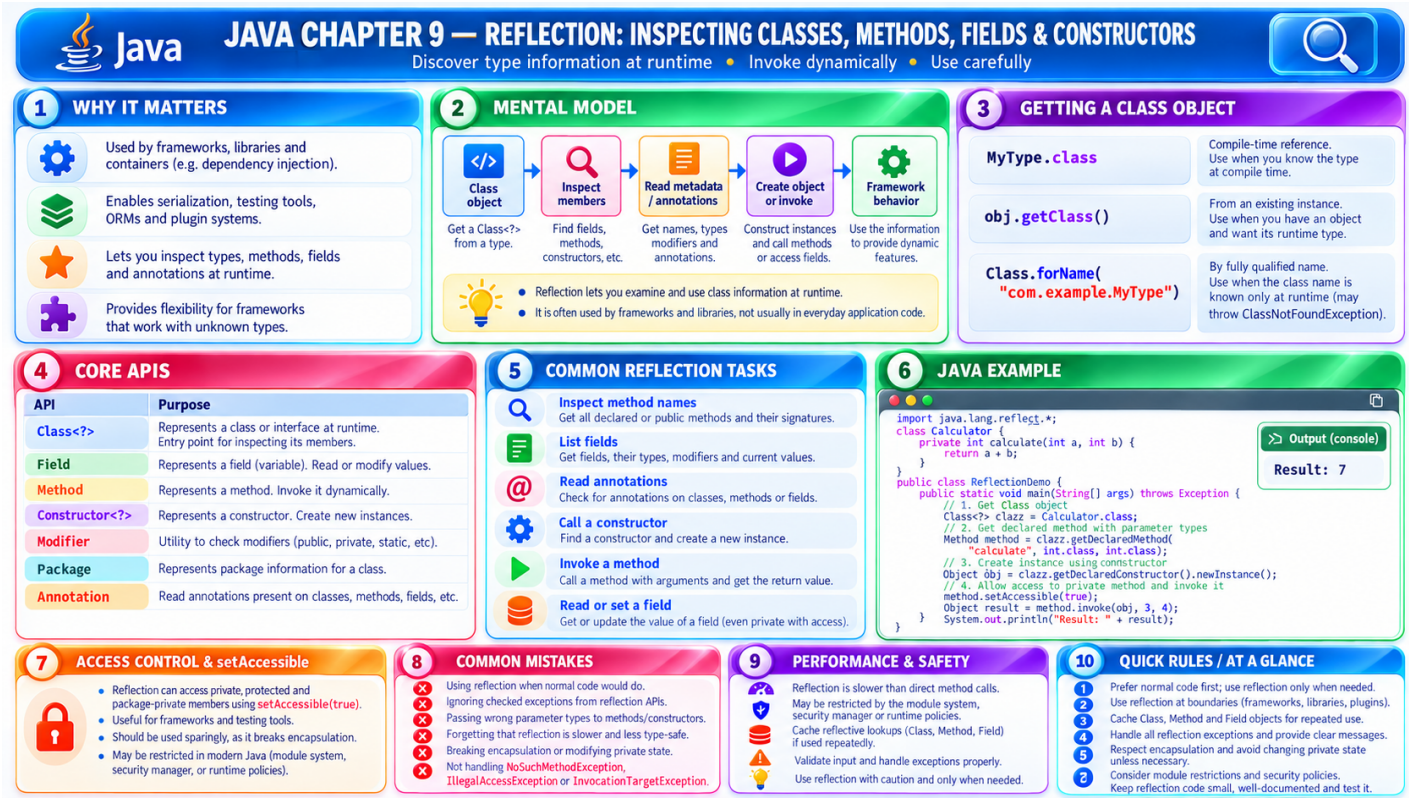
### What can go wrong

- Generating the same file on multiple rounds.
- Forgetting to register the processor or configure build-tool annotation processing.

### Challenge and worked answer

- **Question:** Can a compile-time annotation processor rely on RetentionPolicy.RUNTIME?
- **Answer:** No. Processors work on compiler model elements and can inspect source annotations regardless of runtime retention.

# PART B — REFLECTION



## B1. Why reflection exists

### The idea in ordinary English

- Ordinary Java calls are checked using types known at compile time. Reflection lets code examine names, methods, constructors, and metadata while it runs.
- Frameworks use it for discovery, object construction, mapping, and test execution; business logic usually should not need much of it.

When you read a framework that obtains Class objects, do not assume it already has an instance. Class metadata and an object instance are separate concepts.

### Follow the behavior step by step

- `Customer.class` yields a Class object for a known type.
- `obj.getClass()` obtains the runtime class of an object.
- `Class.forName(...)` can find/load a class by name.
- With Class, you can inspect constructors, fields and methods.

### What can go wrong

- Confusing metadata with an actual Customer object.
- Assuming Class.forName always succeeds or that reflective access is unrestricted.

### Challenge and worked answer

- **Question:** Which object does Customer.class create?
- **Answer:** A Class metadata object reference, not a new Customer business object.

## B2. Inspecting and invoking a class

### The idea in ordinary English

- Use getDeclaredXxx to inspect members declared in one class, and getXxx for public members including inherited ones.
- Constructor.newInstance creates an object; Method.invoke executes a reflected method.

This example contrasts declared fields with an inherited public method, then constructs a Customer and invokes greeting().

### Follow the behavior step by step

- Class metadata says Customer extends BaseEntity.
- getDeclaredFields() counts the private name field.
- getMethod("identity") can see the inherited public method.
- The public constructor creates Customer("Asha").
- The greeting method invocation returns Hello Asha.

### Complete source: ReflectionInspector.java

```
package demo.ch9;
```

```
import java.lang.reflect.*;
class BaseEntity { public String identity() { return "base"; } }
class Customer extends BaseEntity {
    private final String name;
    public Customer(String name) { this.name = name; }
    public String greeting(String prefix) { return prefix + name; }
}
public class ReflectionInspector {
    public static void main(String[] args) throws Exception {
        Class<?> type = Customer.class;
        System.out.println("Class=" + type.getSimpleName());
        System.out.println("Superclass=" + type.getSuperclass().getSimpleName());
        System.out.println("Declared fields=" + type.getDeclaredFields().length);
        System.out.println("Public inherited identity=" + (type.getMethod("identity") != null));
        Object object = type.getConstructor(String.class).newInstance("Asha");
        Method m = type.getMethod("greeting", String.class);
        System.out.println("Invoke=" + m.invoke(object,"Hello "));
    }
}
```

## Verified execution / build result

```
Class=Customer
Superclass=BaseEntity
Declared fields=1
Public inherited identity=true
Invoke=Hello Asha
```

### What can go wrong

- getDeclaredMethod does not search inherited methods.
- Passing wrong parameter types or using inaccessible members produces reflective exceptions.

### Challenge and worked answer

- **Question:** Why does getMethod("identity") succeed even though Customer does not declare identity()?
- **Answer:** Because getMethod searches public methods, including inherited public methods.

## B3. Invocation failures and original causes

**The idea in ordinary English**

- A method invoked through Method.invoke can throw inside the target method.
- Reflection wraps that target exception in InvocationTargetException. You often need getCause() to understand the real failure.

The wrapper is not the root cause. When diagnosing reflection-heavy application code, look for the user method exception underneath it.

**Follow the behavior step by step**

- Processor.run(-1) throws IllegalArgumentException.
- Method.invoke wraps it in InvocationTargetException.
- The catch block calls getCause() and prints the original type and message.

### Complete source: ReflectionFailureDemo.java

```
package demo.ch9;
import java.lang.reflect.*;
class Processor {
    public void run(int count) { if(count<0) throw new IllegalArgumentException("negative count"); }
}
public class ReflectionFailureDemo {
    public static void main(String[] args) throws Exception {
        Method m = Processor.class.getMethod("run", int.class);
        try { m.invoke(new Processor(), -1); }
        catch (InvocationTargetException ex) {
            Throwable cause=ex.getCause();
            System.out.println("Wrapper=" + ex.getClass().getSimpleName());
            System.out.println("Cause=" + cause.getClass().getSimpleName());
            System.out.println("Message=" + cause.getMessage());
        }
    }
}
```

### Verified execution / build result

```
Wrapper=InvocationTargetException
Cause=IllegalArgumentException
Message=negative count
```

**What can go wrong**

- Logging only InvocationTargetException without its cause.
- Swallowing reflective exceptions and losing their debugging context.

**Challenge and worked answer**

- **Question:** If a reflected method throws SQLException, what exception does Method.invoke throw directly?
- **Answer:** InvocationTargetException; its cause is the SQLException.

## B4. Generic metadata and erased runtime types

**The idea in ordinary English**

- Java generics undergo type erasure, but certain generic declarations retain signature information in bytecode.

- Field.getType() returns raw runtime class information; Field.getGenericType() can expose declared type arguments.

Use this when a serializer or mapper wants to know that a field was declared List<String> rather than just List.

**Follow the behavior step by step**

- The declared field is List<String> names.
- getType() returns List as a raw Class.
- getGenericType() returns a ParameterizedType describing List<String>.
- This metadata does not make individual ArrayList elements automatically type-safe at runtime.

### Complete source: GenericsReflectionDemo.java

```
package demo.ch9;
import java.lang.reflect.*;
import java.util.*;
class Bucket { public List<String> names; }
public class GenericsReflectionDemo {
    public static void main(String[] args) throws Exception {
        Field f=Bucket.class.getField("names");
        System.out.println("Raw=" + f.getType().getSimpleName());
        System.out.println("Generic=" + f.getGenericType().getTypeName());
        System.out.println("Parameterized=" + (f.getGenericType() instanceof ParameterizedType));
    }
}
```

### Verified execution / build result

```
Raw=List
Generic=java.util.List<java.lang.String>
Parameterized=true
```

**What can go wrong**

- Assuming reflection can always recover the generic parameter of any runtime object.
- Confusing declared signatures with reified runtime collection types.

**Challenge and worked answer**

- **Question:** Will new ArrayList<String>().getClass() tell you String as a type argument?
- **Answer:** No. The runtime ArrayList class does not retain that instance-specific type argument.

## B5. Access controls, MethodHandles and modules

**The idea in ordinary English**

- Reflection cannot automatically bypass every access rule. Java platform module boundaries can prevent deep access.
- trySetAccessible may return false and setAccessible may throw InaccessibleObjectException.
- MethodHandles is an alternative invocation mechanism with different access and performance characteristics.

Business code should prefer public APIs. If a framework relies on private-field access, configuration, module openness, and security boundaries matter.

**Follow the behavior step by step**

- Start with public constructors and methods when possible.

- If private access is truly needed, check whether the package is open to the caller module.
- Use trySetAccessible when failure is a supported outcome.
- Treat add-opens VM options as deliberate configuration, not a universal fix.

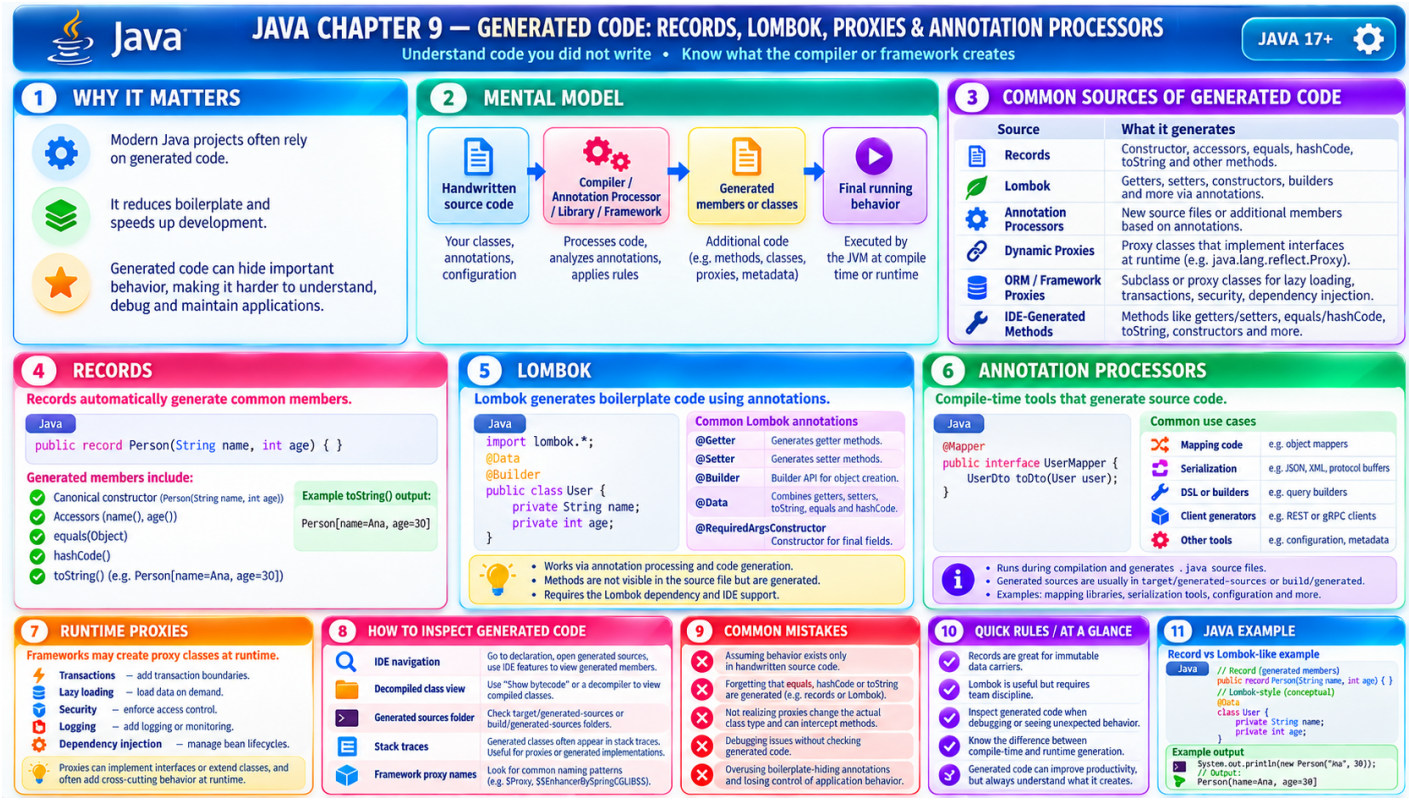
## What can go wrong

- Assuming all private members are accessible just because reflection exists.
- Using setAccessible(true) as the first choice in ordinary application code.

## Challenge and worked answer

- **Question:** Why might a reflection library work on the classpath but fail after moving to named Java modules?
- **Answer:** Strong encapsulation may disallow deep reflection unless the module opens the relevant package.

# PART C — GENERATED CODE AND HIDDEN BEHAVIOR



## C1. Three sources of code you did not type

### The idea in ordinary English

- The Java compiler creates certain members for language constructs such as records.
- Build-time annotation processors can generate new Java source files.
- At runtime, frameworks can create proxies or enhance classes to intercept method calls.

These mechanisms feel similar from the outside but happen at very different times. Debugging becomes easier when you first ask: when was the behavior created?

### Follow the behavior step by step

- Compiler-generated members are part of normal language compilation.
- Annotation processors produce new files in build directories during compilation.
- Lombok transforms compiler-visible structures; its generated methods may not exist as ordinary source files.
- JDK dynamic proxies implement interfaces and forward calls through InvocationHandler.

### What can go wrong

- Assuming Lombok always creates a generated .java file.
- Confusing dynamic proxies with annotation processors.

### Challenge and worked answer

- **Question:** Which mechanism should you investigate when a debugger enters a proxy method before executing a service method?
- **Answer:** Runtime interception or proxying, not ordinary compile-time source generation.

## C2. Records and defensive copying

### The idea in ordinary English

- A record creates a transparent data carrier with components, accessors, a canonical constructor, equals, hashCode and toString.
- A record is shallowly immutable by default; a mutable object referenced by a component can still change.

Record components are final references, but immutability of the object graph takes extra work.

### Follow the behavior step by step

- The constructor validates the id.
- List.copyOf creates a separate unmodifiable snapshot.
- Changing the original input list does not modify the record contents.
- Two Orders with equal components compare equal.

### Complete source: RecordsDemo.java

```
package demo.ch9;
import java.util.*;
```

```
record Order(String id, List<String> items) {
    public Order {
        Objects.requireNonNull(id);
        items = List.copyOf(items);
    }
}
public class RecordsDemo {
    public static void main(String[] args) {
        List<String> input = new ArrayList<>(List.of("Book"));
        Order a = new Order("A-1",input);
        Order b = new Order("A-1",List.of("Book"));
        input.add("Mouse");
        System.out.println("Value equality=" + a.equals(b));
        System.out.println("Items=" + a.items());
        try { a.items().add("Pen"); }
        catch (UnsupportedOperationException ex) { System.out.println("Items cannot be modified"); }
    }
}
```

## Verified execution / build result

```
Value equality=true
Items=[Book]
Items cannot be modified
```

### What can go wrong

- Assuming final references make all nested objects immutable.
- Returning a mutable list as a record component without copying it.

### Challenge and worked answer

- **Question:** Why would record Order(String id, List<String> items) not be deeply immutable without a defensive copy?
- **Answer:** Because clients could mutate the original list after record construction or mutate the list returned by items().

# C3. A real annotation processor that generates source

### The idea in ordinary English

- This is compile-time code generation, not just a description of it.
- The included processor finds @GenerateHello and creates demo.ch9.GeneratedHello.java during javac compilation.

The generated class can then be compiled and referenced like any other Java class. This is how build-time tooling creates boilerplate or mapping code that you do not see among handwritten files.

### Follow the behavior step by step

- Compile the annotation definition and processor.
- Run javac with -processor demo.ch9.AnnotationProcessorDemo and -s to select generated-source location.
- The Filer writes GeneratedHello.java.
- javac processes and compiles the generated class.

## Complete source: AnnotationProcessorDemo.java

```
package demo.ch9;
import java.lang.annotation.*;
import javax.annotation.processing.*;
import javax.lang.model.SourceVersion;
import javax.lang.model.element.*;
import javax.tools.JavaFileObject;
import java.io.Writer;
import java.util.Set;
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.SOURCE)
@interface GenerateHello { }
@SupportedAnnotationTypes("demo.ch9.GenerateHello")
@SupportedSourceVersion(SourceVersion.RELEASE_17)
public class AnnotationProcessorDemo extends AbstractProcessor {
    @Override public boolean process(Set<? extends TypeElement> annotations, RoundEnvironment round) {
        if (round.processingOver()) return false;
        for(Element e:round.getElementsAnnotatedWith(GenerateHello.class)) {
            try {
                JavaFileObject file=processingEnv.getFiler().createSourceFile("demo.ch9.GeneratedHello",e);
                try(Writer writer=file.openWriter()) {
                    writer.write("package demo.ch9; public class GeneratedHello { public static String message() { return \"Generated a
                }
            } catch(Exception ex) { throw new IllegalStateException(ex); }
        }
        return true;
    }
}
```

## Verified execution / build result

```
Annotation processor successfully generated demo.ch9.GeneratedHello.java with method message() returning: 
```

### What can go wrong

- Running the processor without registering it.
- Creating the same generated type more than once.
- Confusing generated output folders with source-control files.

### Challenge and worked answer

- **Question:** What do you inspect if an annotation-driven mapper interface has no implementation source in src/main/java?

• **Answer:** Look in build/generated-sources or the compiler output, then confirm the processor ran and that generated classes were compiled.

## Complete source: AnnotationProcessorDemo.java

```
package demo.ch9;
import java.lang.annotation.*;
import javax.annotation.processing.*;
import javax.lang.model.SourceVersion;
import javax.lang.model.element.*;
import javax.tools.JavaFileObject;
import java.io.Writer;
import java.util.Set;
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.SOURCE)
@interface GenerateHello { }
@SupportedAnnotationTypes("demo.ch9.GenerateHello")
@SupportedSourceVersion(SourceVersion.RELEASE_17)
public class AnnotationProcessorDemo extends AbstractProcessor {
    @Override public boolean process(Set<? extends TypeElement> annotations, RoundEnvironment round) {
        if (round.processingOver()) return false;
        for(Element e:round.getElementsAnnotatedWith(GenerateHello.class)) {
            try {
                JavaFileObject file=processingEnv.getFiler().createSourceFile("demo.ch9.GeneratedHello",e);
                try(Writer writer=file.openWriter()) {
                    writer.write("package demo.ch9; public class GeneratedHello { public static String message() { return \"Generated a
                }
            } catch(Exception ex) { throw new IllegalStateException(ex); }
        }
        return true;
    }
}
```

## Complete source: GenerateHelloUsage.java

```
package demo.ch9;
@GenerateHello
class GenerateHelloUsage { }
```

The project package contains the actual generated Java source under `generated-sources/demo/ch9/GeneratedHello.java`. The program output file records this build result.

## C4. Dynamic proxies and invocation interception

**The idea in ordinary English**

- A JDK dynamic proxy implements one or more interfaces at runtime.
- A caller invokes a method on the proxy; its InvocationHandler runs first and may delegate to the real implementation.

This pattern helps explain cross-cutting behavior such as logging, transactions, authorization and timing. Spring may use JDK interface proxies or class-based proxies depending on configuration.

**Follow the behavior step by step**

- Create a target GreetingService implementation.
- Create a proxy implementing GreetingService.
- The handler prints Before greet and delegates via reflection.
- The target returns Hello Riya, which the caller prints.

### Complete source: DynamicProxyDemo.java

```
package demo.ch9;
import java.lang.reflect.*;
interface GreetingService { String greet(String name); }
class SimpleGreeting implements GreetingService {
    public String greet(String name) { return "Hello " + name; }
}
public class DynamicProxyDemo {
    public static void main(String[] args) {
        GreetingService target = new SimpleGreeting();
        GreetingService proxy = (GreetingService) Proxy.newProxyInstance(
            GreetingService.class.getClassLoader(), new Class<?>[]{GreetingService.class},
            (instance, method, arguments) -> {
                System.out.println("Before " + method.getName());
                try { return method.invoke(target,arguments); }
                catch(InvocationTargetException ex) { throw ex.getCause(); }
            });
        System.out.println(proxy.greet("Riya"));
    }
}
```

### Verified execution / build result

```
Before greet
Hello Riya
```

**What can go wrong**

- Assuming a JDK interface proxy is the same class as the target.
- Forgetting to preserve the underlying exception cause.
- Calling internal methods on the target and expecting every call to traverse the proxy.

**Challenge and worked answer**

- **Question:** Why might a method called from another method on the same Spring bean bypass advice in proxy-based AOP?
- **Answer:** Because self-invocation often uses this rather than going through the proxy reference.

## C5. Lombok: generated methods and when not to trust assumptions

### The idea in ordinary English

- Lombok reduces source boilerplate using annotations such as @Getter, @Setter, @Builder and @RequiredArgsConstructor.
- Its behavior depends on annotation processor integration, plugin support, and annotation configuration.

If you see a Lombok class with no written constructor, check which annotation could have generated one before assuming the code cannot compile.

### Follow the behavior step by step

- Check the project pom.xml or Gradle dependencies for Lombok.
- Use the IDE or delombok to inspect the effective generated methods.
- Verify generated equality and toString behavior for sensitive or mutable fields.
- Be aware that generated constructors, accessors and builders alter the API surface.

### What can go wrong

- Assuming @Data is harmless on entities with relationships.
- Assuming generated equals/hashCode always suits mutable entities.

### Challenge and worked answer

- **Question:** What is the first investigation when code calls getName() but no getName() appears in the class?
- **Answer:** Check Lombok or other source generators before concluding the method is missing.

# Putting all three parts together

Imagine a Spring Boot application with a class annotated `@Service`, a dependency injected into its constructor, JSON mapped to a record, and a method wrapped in transaction handling. A useful debugging path is:

**Trace the hidden behavior**

- Annotation: who interprets @Service or @Transactional? The Spring container and proxy infrastructure, not the annotation itself.
- Reflection/metadata: how does the framework discover constructors, fields, or record components?
- Generated code: did a mapper, serializer, or Lombok transform code at build time?
- Runtime proxy: is a transactional call intercepted? Does the call go through the proxy?
- Failure: unwrap exceptions and identify the original cause before fixing anything.

This mental model is more valuable than memorizing every annotation or reflective API. It tells you where to look when unfamiliar framework code appears to do something without a visible method call.

# High-value API cheat sheet

| Concept | Read / investigate | Important limitation |
|---|---|---|
| Retention | @Retention(RUNTIME) | Only runtime-retained metadata is visible through ordinary runtime reflection |
| Declared method | getDeclaredMethod(...) | Looks only at declared methods |
| Public method | getMethod(...) | Includes inherited public methods |
| Original failure | InvocationTargetException.getCause() | Reflection wraps underlying target exception |
| Generic field | Field.getGenericType() | Declared signature, not fully reified instances |
| Access | trySetAccessible() | May fail under strong encapsulation |
| Source generation | AbstractProcessor + Filer | Occurs during compilation, must be configured |
| JDK proxy | Proxy.newProxyInstance | Requires an interface |
| Record | record Order(...) | Only shallowly immutable unless defensive copy made |

# Review exercises — worked solutions

**1. Why is a @Retention(CLASS) annotation invisible to getAnnotation at runtime?**

Answer: CLASS-retained metadata is recorded in bytecode but not retained for ordinary runtime reflection.

**2. How would you filter methods with a custom annotation but avoid accidental invocation?**

Answer: Use getDeclaredMethods; check getAnnotation; validate public access, signature, and expected arguments; only then invoke.

**3. What is the difference between Class.forName and new Customer()?**

Answer: Class.forName resolves/loads the class; new creates an object by running a constructor.

**4. Why is InvocationTargetException usually not the business failure itself?**

Answer: It wraps the exception thrown by the reflected target method; inspect getCause().

**5. What does getGenericType() tell you that getType() does not for List field?**

Answer: It can expose List from the declared field signature rather than raw List alone.

**6. How does a source processor differ from JDK Proxy?**

Answer: One generates source during compilation; the other creates a runtime interface implementation.

**7. Why does List.copyOf matter in records?**

Answer: It gives the record a stable snapshot and an unmodifiable list rather than a live mutable reference.

**8. Which build directory would you inspect for a missing generated mapping implementation?**

Answer: The generated-sources or annotation-processing output directory, then the compiled classes.

**9. What are the implications of proxy self-invocation?**

Answer: The call may bypass proxy advice, so transaction/logging behavior may not apply.

**10. When should you avoid reflection in application code?**

Answer: When direct interfaces, constructors, ordinary dispatch, or existing framework APIs solve the problem more clearly and safely.

# Project guide and build verification

All Java samples are complete files under `src/demo/ch9/`. They were compiled using `javac --release 17` and executed successfully.

**Recommended run order**

- AnnotationDemo and AnnotationRulesDemo — start with metadata and retention.
- ReflectionInspector, ReflectionFailureDemo, GenericsReflectionDemo — inspect classes and diagnose failures.
- RecordsDemo — understand compiler-generated behavior and shallow immutability.
- DynamicProxyDemo — observe invocation interception.
- AnnotationProcessorDemo — reproduce real compile-time source generation.

Compile ordinary demos: `javac --release 17 -d classes src/demo/ch9/AnnotationDemo.java ...` (or import the complete source directory into IntelliJ). The README has exact processor commands, because processor compilation happens in two passes.

Depth audit was applied to coverage of annotation lifecycle, retention and meta-annotations, runtime and compile-time processing, reflection APIs and access, errors, generic signatures, records, Lombok, generated sources, proxies, and practical diagnostics.

## Deep dive 1 — Metadata lifetimes, exactly

**What people get wrong**

An annotation is a class-file metadata feature only when its chosen retention says so. SOURCE annotations are available to source-oriented tools and annotation processors; CLASS ones are stored in the class file but not visible to ordinary runtime reflection; RUNTIME annotations survive and can be queried in a running program.

**Worked trace**

A developer writes @Retention(CLASS) @interface Audit. After javac, Audit metadata may remain in the .class file. Calling SomeClass.class.getAnnotation(Audit.class) returns null. Changing retention to RUNTIME, recompiling, and running the same call makes the annotation available.

**Diagnostic sequence**

Ask: (1) Does this annotation target the correct element? (2) Is it retained to the phase that reads it? (3) Which tool actually scans it? (4) Is that scanning tool configured? Do not assume the presence of the annotation proves it is being processed.

**Try it and answer**

Predict: a SOURCE-retained annotation on a method is read with getDeclaredMethods and getAnnotation at runtime. Answer: the reflected method is visible but the annotation will not be returned.

## Deep dive 2 — Annotation inheritance edge cases

**What @Inherited does**

When a class lookup asks for an annotation that is itself marked @Inherited, Java can look through superclasses. That does not mean annotations on interface declarations are inherited, nor does it mean methods automatically inherit annotations from overridden methods.

**Repeatable annotations**

@Repeatable(Tags.class) means multiple @Tag annotations can be grouped in a container. The container must declare Tag[] value(), and its retention/target settings must comply with the repeated annotation constraints.

**Real debugging scenario**

A framework queries getDeclaredAnnotation and does not see a base-class annotation; another piece of code uses getAnnotation and does. The different lookup contract, not a broken compiler, explains the discrepancy.

**Try it and answer**

Why is @Inherited not enough to make an @Action method in the superclass visible via Child.class.getDeclaredMethods? Answer: getDeclaredMethods only enumerates methods declared directly on Child; method annotations do not use class-level @Inherited propagation.

## Deep dive 3 — Processor rounds and source generation

**The compilation lifecycle**

javac first loads an annotation processor, scans source model elements, calls process, and may run new processing rounds when new source files are created. A processor may run again even when no new matching source annotations are present, including a final processingOver round.

**The right extension points**

Use ProcessingEnvironment for services, RoundEnvironment to access annotated elements, Elements/Types utilities for model reasoning, Messager for diagnostics, and Filer to create generated source files. The elements are compiler model objects, not runtime java.lang.reflect.Method objects.

**Failure diagnosis**

A FilerException can result when the same source file is generated twice. An unregistered processor creates nothing. A generated source file that fails compilation may leave no usable class. Inspect javac diagnostics and generated-sources separately from compiled class output.

**Try it and answer**

Why does our processor test run javac twice? Answer: first it compiles the processor and its annotation, then a second javac invocation loads that processor to process the annotated usage class.

## Deep dive 4 — Reflection lookup precision

**Declared versus public lookups**

getDeclaredMethods exposes declared methods at all visibility levels; getMethods returns public methods including inherited public methods. getDeclaredConstructor does not search the hierarchy; constructors are never inherited.

**Invocation behavior**

Method.invoke takes an object receiver for instance methods, plus argument values. It can fail due to absent method names, incorrect argument types, inaccessible targets, or a target method exception wrapped in InvocationTargetException.

**Worked mental trace**

Customer.class.getMethod("greeting", String.class) obtains metadata. Customer.class.getConstructor(String.class).newInstance("Asha") creates a target instance. invoke(instance, "Hello ") then dispatches the method and yields a String. Nothing happens at method lookup time.

**Try it and answer**

What happens if greeting(String) is made private and the code keeps getMethod? Answer: getMethod does not locate private methods; use getDeclaredMethod with an access strategy that the module system permits.

## Deep dive 5 — Runtime type metadata and generics

**Two levels of type**

At runtime, both ArrayList and ArrayList normally share the same Class object. Yet a declared field List can retain its generic signature in reflection metadata, available via getGenericType.

**What the signature really guarantees**

It tells you the declaration, not the historical element type of an arbitrary runtime list. Unchecked casts or raw types can still violate your assumptions. A serializer needs careful treatment of ParameterizedType, GenericArrayType, TypeVariable, and WildcardType.

**A practical boundary**

For an API accepting List as a variable, type erasure means you cannot just ask list.getClass().getGenericType() to recover a concrete T. Frameworks often accept explicit type tokens or inspect field/method declarations.

**Try it and answer**

Can you discover the T in a raw new ArrayList() based only on the object instance? Answer: not reliably. The instance class normally does not carry the specific source type argument.

## Deep dive 6 — Strong encapsulation, access, and MethodHandles

**Why accessible may fail**

setAccessible(true) does not promise permission. Named modules enforce strong encapsulation, and JDK packages are not necessarily open to your unnamed or named module. trySetAccessible() gives a non-throwing boolean for some access failures.

**What to do in a project**

Prefer constructors, public APIs, or documented framework hooks. If a library requires private access, inspect module-info.java and any open-package configuration. Do not scatter arbitrary --add-opens flags without understanding the boundary.

**MethodHandles**

MethodHandles.Lookup supplies typed method and field handles. It can offer flexible invocation without constructing Method objects, but its lookup permissions also obey access rules. It is not an unrestricted backdoor into private implementation.

**Try it and answer**

A library calls setAccessible(true) on a JDK-internal field and receives InaccessibleObjectException. Is ignoring the exception safe? Answer: no; the access was not granted. Investigate supported APIs or explicit module openness.

# Deep dive 7 — Lombok, records, and generated API contracts

**Records are real language constructs**

A record generates component accessors, equals, hashCode and toString, plus canonical constructor behavior. Records are final and intended as transparent carriers; the component references are final, but referenced collections may be mutable.

**Lombok is different**

Lombok modifies the compiler model / generated bytecode through tooling integration. You may not see an ordinary generated Java file. @Data affects getters, setters, equals, hashCode and toString; for entities and mutable objects these choices deserve scrutiny.

**Correct inspection path**

For records, inspect the language contract and any compact/canonical constructor. For Lombok, inspect annotations, dependencies and configuration; use IDE support or delombok to see generated shape. For source processors such as MapStruct, inspect generated source folders.

**Try it and answer**

Why can a record containing List still change in observable behavior? Answer: if the list reference points to a mutable list that other code holds or if the accessor exposes it, list contents can change unless copied.

## Deep dive 8 — Proxy call paths, framework advice, and debugging

**JDK proxy call path**

Caller -> interface proxy -> InvocationHandler -> target invocation -> return or throw. The handler may add logging, authorization or metrics before delegating. It must preserve results, exceptions and object semantics appropriate to the interface.

**Spring proxy caveat**

Spring AOP can employ interface-based or class-based proxies. Advice applies at the proxy interception boundary. A direct self-call on this may bypass that boundary, explaining why an apparently annotated method does not receive transaction or security advice.

**When to inspect generated types**

Log proxy.getClass(), compare it to the declared interface and actual target. Investigate the framework proxy configuration and invocation entry point. Do not treat the runtime proxy class as the original service class.

**Try it and answer**

Why does our example print Before greet before Hello Riya? Answer: the proxy interceptor runs before Method.invoke delegates to the real service.